

GhostPrint

Clone-resistant repeat-fraudster re-identification

Project Guide & Study Notes

github.com/virinchisai/ghostprint

GhostPrint — Project Guide & Study Notes

A complete walkthrough of the project: the problem, the concepts you need to understand it, every design decision and why it was made, the results and how to read them, the limitations, and a bank of technical questions with answers. Read this top to bottom and you can defend any part of the project.

1. The problem in one paragraph

Banks and contact centers keep **voice watchlists** of known fraudsters. When a caller comes in, the system compares their **voiceprint** (a numerical fingerprint of the sound of their voice) against the list; a match escalates the call. This works until a fraudster uses a **voice changer or AI voice clone** — then the sound of the voice, the exact thing the voiceprint measures, is replaced, and the match silently fails. Existing deepfake detectors answer *"is this voice fake?"* but not the operational follow-up: *"and is the person behind it the same fraudster who hit us last week?"* GhostPrint answers that second question.

Why Pindrop cares: their 2025 Voice Intelligence report cites a ~1,300% surge in deepfake voice fraud — exactly the trend that erodes the voiceprint watchlists their customers depend on. GhostPrint is the defensive complement to their Pulse deepfake detector: Pulse flags the fake, GhostPrint re-identifies the fraudster behind it.

2. The core idea

A voice clone changes **what you sound like** (timbre) but not **how you speak**: the melody of your sentences, your rhythm, where and how long you pause, your speaking rate. Think of it as **gait recognition for the voice** — a disguise hides your face, but you still walk like you.

GhostPrint builds an identity fingerprint from those **clone-surviving** signals and is **trained to be invariant to disguise**: during training it sees the same speaker both normal and voice-altered and is forced to map both to the same point in embedding space. The result is a fingerprint that barely moves when the voice is changed.

Hypothesis (falsifiable): a speaker embedding built from prosody/rhythm and trained with disguise augmentation will keep its identification accuracy under voice disguise, whereas a timbre voiceprint will degrade.

3. Background concepts (know these cold)

Speaker embedding / voiceprint. A neural network maps an utterance to a fixed-length vector (here 192 numbers) such that the same speaker's utterances land close together and different speakers land far apart. "Close" is measured by **cosine similarity** (dot product of L2-normalized vectors, range $-1 \dots 1$).

Timbre vs. prosody.

- **Timbre** = the spectral/tonal color of the voice (formants = vocal-tract

resonances, spectral envelope). This is what ECAPA and most voiceprints encode, and what voice conversion rewrites.

- **Prosody** = the supra-segmental patterns: pitch contour (intonation), rhythm,

stress, pauses, tempo. Carries weaker but real speaker identity, and survives timbre manipulation.

ECAPA-TDNN. The current standard speaker-verification architecture (Emphasized Channel Attention, Propagation and Aggregation Time-Delay Neural Network). We use the pretrained [speechbrain/spkrec-ecapa-voxceleb](#) model as the **baseline** — it represents "what watchlists use today." We do not train it; it's the incumbent we measure against.

Evaluation metrics.

- **Rank-1 identification** (%): given a probe, rank all enrolled speakers by similarity; is the true speaker #1? Higher is better. This is the "identification" or watchlist-lookup view.
- **EER, Equal Error Rate** (%): sweep the accept/reject threshold; the point where the **false-accept rate** equals the **false-reject rate**. Lower is better. This is the "verification" (yes/no match) view, threshold-independent.
- **Hit-rate @ 1% FAR** (%): fix a strict false-alarm budget (1% of impostors wrongly flagged) and measure how many true repeat offenders you catch. This is the metric a real fraud team lives by — you can only tolerate so many false accusations. Higher is better.

Open-set vs closed-set. **Closed-set** assumes every probe belongs to an enrolled speaker. **Open-set** (what we do) allows probes from people not on the list — which is the real watchlist situation. Our 40 evaluation speakers are **disjoint** from the 38 training speakers, so we measure generalization to people the model never saw.

Enrollment / probe. **Enrollment** = the reference material on file (here 5 clean utterances per speaker → averaged into a **centroid**, the "fraudster file"). **Probe** = a new incoming call scored against the centroids.

AAM-Softmax (ArcFace). A classification loss that adds an **angular margin**: it forces each speaker's embeddings not just to be correct but to sit a safety margin away from other speakers on the unit hypersphere. This produces embeddings that cluster tightly by identity and generalize to unseen speakers — the standard choice for training speaker embeddings.

PSOLA / voice disguise. Pitch-Synchronous Overlap-Add: a classic DSP method (via Praat's "Change gender") that resynthesizes speech with shifted **pitch**, scaled **formants**, and altered **duration**. We use it to **simulate** a fraudster masking their voice. It moves the same axes a neural clone does, but is weaker (see Limitations).

Score normalization (s-norm) and fusion. Raw cosine scores from different systems aren't directly comparable. **Cohort z-normalization** rescales each probe's score by the mean/std of its scores against all **other** speakers, making scores comparable across probes and systems. **Fusion** then simply adds the two systems' normalized scores (equal weight, no tuning) — a deliberately simple, honest combiner.

4. Method, component by component

4.1 Data ([ghostprint/data.py](#), [scripts/prepare_data.py](#))

- **LibriSpeech** (public read-speech corpus). `dev-clean` → training

(38 speakers after keeping utterances 4–20 s long); `test-clean` → evaluation (40 speakers, unseen).

- Per training speaker: 30 utterances. Per eval speaker: 5 enrollment + 12 probe.
- For each utterance we synthesize **disguised copies** (see 4.2). Training gets

2 random presets/utterance; eval probes get all disguise conditions.

4.2 Disguise simulation (`ghostprint/disguise.py`)

- `apply_disguise(in, out, preset)` uses Praat "Change gender" (PSOLA) with a preset = (pitch semitones, formant ratio, duration factor).
- Discrete presets: `pitch_up/down` (± 3 st), `gender_up/down` (formant 1.14/0.88 + pitch), and `hard` (formant 1.10, -2 st, tempo 0.94) — `hard` is never seen in training, so it tests generalization to an unseen disguise.
- `SWEEP_PRESETS`: formant 0.96→0.65 with pitch held near-constant, for the dose-response curve.
- `apply_neural_vc(...)` is a documented **hook**: swap in RVC/FreeVC/kNN-VC/XTTS to rerun the identical protocol against real neural cloning (needs a GPU).

4.3 Clone-surviving features (`ghostprint/features.py`) — the heart of the idea

Every feature is **relative by construction**, so shifting absolute pitch/formants (what a clone/disguise does) moves it as little as possible:

- **Frame features** (50 fps, 4 dims): `zlog_f0` (log-F0 z-scored *within the utterance* → invariant to absolute pitch level), `voiced` flag, `z_energy`, `d_zlog_f0` (delta = intonation *movement*).
- **12 utterance statistics**, all pitch-level-independent: pause rate, mean/std/max pause duration, voiced fraction, pseudo-syllable rate (energy peaks), mean/std voiced-run length, F0 range and std **in semitones relative to the speaker's own median**, F0 slope (pitch movement speed), energy modulation.

The design principle: encode *shape and rhythm*, never absolute frequency. That's why the GhostPrint line stays flat when formants are scaled.

4.4 Model (`ghostprint/model.py`)

- **ProsodyEncoder**: a 2-layer **bidirectional GRU** over the frame features (captures temporal intonation/rhythm patterns), with **mean + max pooling** over time; the 12 stats go through a small MLP; both are concatenated and projected to a **192-d L2-normalized embedding** (same dimensionality as ECAPA, so fusion is symmetric).
- **AAMSoftmax head** (margin 0.2, scale 30) used only during training.

4.5 Training (`scripts/train.py`) — where invariance is learned

- Each sample: pick an utterance, then with probability `disguise_prob = 0.6`

substitute one of its **disguised** variants. So the same speaker label is attached to clean *and* disguised versions → the network is pushed to make them collapse to one embedding. **This augmentation is the invariance mechanism.**

- Adam, lr 1e-3, cosine schedule, batch 64. Random 8-second crops + tiny feature noise for regularization. Checkpoint every epoch.

- Ran 15 epochs on CPU (originally configured for 40; reduced for time — see the "why 15" question below). Train accuracy rose 3% → 41%; note this is measured *with* the angular margin active and on 60%-disguised samples, so it understates the easier eval-time scoring.

4.6 Baseline & scoring (`scripts/evaluate.py`)

- **ECAPA** embeddings from the pretrained model (not trained by us).
- For each system: build 5-utterance clean **centroids**; score probes by cosine to each centroid; apply **cohort z-norm**; compute EER / rank-1 / hit-rate@1%FAR.
- **Fusion** = equal-weight sum of the two systems' z-normed scores. No eval-set tuning anywhere.

4.7 Behavioral channel (`scripts/ivr_experiment.py`)

- A **simulated** IVR channel: 60 synthetic "fraudsters" with stable habits (menu-path Markov transitions, log-normal inter-key timing, input-error rate). Re-identify them across sessions from behavior alone — no audio.
- Clearly labeled simulation. Its purpose: show the fusion architecture extends to the metadata/behavioral telemetry a real IVR already logs — a channel a voice clone cannot touch at all.

4.8 Investigator demo (`demo.py`)

- `python demo.py callA.wav callB.wav` prints the timbre-voiceprint cosine and the prosody-fingerprint cosine and a verdict. The key case: **voiceprint says "different," prosody says "same" → likely a disguised repeat fraudster**. That cross-channel *disagreement* is itself a detection signal.

5. Results and how to read them

5.1 Discrete conditions (40 unseen speakers), Rank-1 %

condition	ECAPA	GhostPrint	Fusion
clean	100.0	29.8	100.0
pitch_up	100.0	29.8	100.0
pitch_down	100.0	27.5	99.4

condition	ECAPA	GhostPrint	Fusion
gender_up	83.5	29.4	87.5
gender_down	74.2	25.4	84.2
hard (unseen)	97.3	26.9	96.3

Watchlist hit-rate @1% FAR tells the same story: gender_down 79.2 (ECAPA) → **84.6** (fusion).

5.2 Formant dose-response sweep — the headline

formant ratio	ECAPA rank-1	GhostPrint rank-1	ECAPA EER	GhostPrint EER
1.00	100.0	29.8	0.0	19.8
0.96	100.0	29.4	0.0	19.4
0.88	86.9	29.8	4.0	19.6
0.80	42.3	28.5	15.6	20.8
0.72	15.8	27.9	31.0	20.4
0.65	13.3	27.5	36.2	20.3

5.3 Behavioral channel (simulated)

60 fraudsters: **82.2% rank-1, 17.7% EER** from IVR interaction patterns alone.

5.4 The four honest takeaways

1 GhostPrint is disguise-invariant — flat across every condition

(rank-1 25–30%, EER ~20%). The core hypothesis holds.

1 Prosody is a weak **absolute** biometric — on clean audio ECAPA crushes it

(100% vs 30%). GhostPrint's value is **stability**, not peak accuracy → it's a complement, not a replacement.

1 The voiceprint collapses under formant distortion — 100% → 13% rank-1;

EER 0% → 36% — and the two systems **cross over at formant $\approx$ 0.76**. Below the crossover, prosody is the **more reliable** identifier. That regime is exactly what a formant-restructuring neural clone creates.

1 Fusion already helps — precisely where ECAPA weakens (gender presets),

fusion beats ECAPA alone (74→84% rank-1). Best of both: clean-audio precision plus a robustness floor under attack.

6. What's novel (be precise, don't overclaim)

The **individual ingredients are known** (prosody features, ECAPA, ArcFace, augmentation). The contribution is the **assembly and framing**:

1 Reframing the task as **re-identification **through** disguise** ("who is behind

the fake voice?"), not deepfake *detection* ("is it fake?").

1 **Training explicitly for disguise-invariance** via clean↔disguised augmentation — the mechanism that produces the flat curve.

1 Evaluating as an **open-set fraud watchlist** (unseen speakers, clean-enroll / disguised-probe, a held-out disguise type, hit-rate@1%FAR) — the protocol *is* the business problem.

1 The **quantified crossover** (formant ≈ 0.76): a measured statement of *where* the voiceprint stops being trustworthy — not found in the public work searched.

1 **Cross-channel disagreement as an alarm** and extension to a **non-audio behavioral channel**.

Prior-art neighbors to cite (shows rigor): classic prosodic-feature speaker recognition; "Catch You and I Can" (recovers source voiceprint from converted audio — purely acoustic); ASVspoof deepfake countermeasures (detection, not re-ID); voiceprint-clustering fraudster-exposure patents (defeated by cloning).

7. Limitations (state these before you're asked — it reads as rigor)

- **PSOLA $\neq$ neural voice conversion.** Same axes, weaker adversary. The sweep is a *controlled lower bound*; the real test needs RVC/XTTS via the `apply_neural_vc` hook (GPU). This is the single most important caveat.

- **Read speech, not calls.** LibriSpeech is audiobooks. Conversational/telephony data (Fisher, real call audio) would strengthen the rhythm/pause channel and unlock an **idiolect** (word-choice) channel — deliberately excluded here because on audiobook text it would leak *book content*, not speaker identity.

- **Scale.** 38+40 speakers is demo scale; the protocol scales to VoxCeleb-size pools by editing `config.py` only.

- **Prosody is inherently weaker than timbre.** GhostPrint raises the *cost* of evasion (a fraudster must now defeat several independent channels at once); it doesn't make evasion impossible. That cost-raising is the point.

- **Short training run** (15 epochs, CPU). More epochs / more speakers would lift GhostPrint's absolute numbers; the invariance result (flatness) is the robust finding regardless.

8. Anticipated technical questions (with answers)

Q: 30% rank-1 sounds terrible — is prosody useless? A: On clean audio, yes it's far weaker than timbre, and I never claim otherwise. The point is *stability*: it's ~28% whether the voice is untouched or heavily disguised, while ECAPA falls from 100% to 13%. Below the crossover it's the better of the two, and fused it adds a robustness floor. It's a complementary

channel, not a replacement voiceprint.

Q: Isn't PSOLA a strawman? Real attackers use neural cloning. A: Correct, and I flag it as the top limitation. PSOLA moves the same axes (pitch/formant/tempo) but is weaker. I built `apply_neural_vc` as a drop-in so the identical protocol runs against RVC/XTTS on a GPU. My expectation: neural VC degrades ECAPA **more** (it restructures formants more thoroughly), which only widens GhostPrint's relative advantage. The current numbers are a conservative lower bound.

Q: Why does GhostPrint stay flat — are you sure it's not a bug? A: By construction. Every feature is relative (log-F0 z-scored within the utterance, statistics in semitones relative to the speaker's own median), so scaling absolute pitch/formants barely perturbs the inputs. Augmentation then trains that invariance in. The flat line across a disguise it never trained on (*hard*, and the 0.65/0.72 sweep points) is the evidence it generalizes, not memorizes.

Q: Why AAM-softmax instead of a contrastive/triplet loss? A: AAM-softmax is the current SOTA default for speaker embeddings — stable to train, strong open-set generalization, no hard-negative mining. A supervised contrastive variant with (clean, disguised) positive pairs is a natural next experiment and would likely sharpen invariance further.

Q: Why is the baseline pretrained but GhostPrint trained from scratch — is that fair?

A: It's deliberately **conservative against my own method**. ECAPA is a strong model pretrained on VoxCeleb (thousands of speakers); GhostPrint is trained from scratch on 38. I'm giving the incumbent every advantage and still showing it collapses under formant distortion while mine holds. A fairer-to-GhostPrint setup (pretrain on more speakers) would only help my side.

Q: Why 15 training epochs? A: Pure compute budget on a CPU laptop — 40 epochs was configured but too slow, so I early-stopped at 15 once the loss curve was still improving. It slightly understates GhostPrint's absolute accuracy. The **invariance** result (flatness across conditions) is independent of run length, which is the claim I actually make.

Q: Could a fraudster beat GhostPrint too? A: Yes — by changing **how** they speak (rate, phrasing, deliberate pausing), not just how they sound. But that's much harder to sustain naturally over a whole call than clicking a voice-changer preset, and it must be done simultaneously with any timbre disguise. The system raises the **cost** and **skill** required, and multi-channel fusion (audio prosody + behavior + metadata) means they must defeat several independent signals at once.

Q: How would this run in production at Pindrop? A: As a scoring add-on to the existing watchlist: alongside the ECAPA voiceprint score, compute the prosody score; fuse; and specifically surface **high-prosody / low-voiceprint** callers as "possible disguised repeat offender" for analyst review. It reuses the same enrollment data and adds one lightweight embedding model. The behavioral channel plugs into IVR telemetry they already collect.

Q: What's your single most convincing number? A: At formant 0.65 the voiceprint is at 13% rank-1 and 36% EER — worse than useless for identity — while GhostPrint is unchanged at 27.5% / 20%. The watchlist has gone blind and the prosody channel hasn't noticed the disguise.

9. How to run it

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
scripts/download_data.sh      # LibriSpeech dev-clean + test-clean (~680 MB)
scripts/run_all.sh           # prepare → features → train → evaluate → IVR
python scripts/sweep.py      # formant dose-response curve (headline figure)
python demo.py a.wav b.wav   # two-call "same person?" comparison
```

CPU end-to-end $\approx$ 2–3 h (most of it ECAPA inference). Outputs land in `results/`. Pinned deps that matter: `speechbrain==1.0.0`, `huggingface_hub==0.25.2`; no `librosa` (its `llvmlite` dep won't build on Intel macOS).

10. 60-second verbal pitch

"Voice watchlists catch repeat fraudsters by matching a voiceprint — the sound of the voice. Voice cloning breaks that, because the sound is exactly what the clone replaces. GhostPrint re-identifies the fraudster from what the clone *doesn't* change — their intonation, rhythm, and pauses — using an embedding trained to be invariant to voice disguise. On an open-set watchlist test, as I push formant distortion up, the standard voiceprint collapses from 100% to 13% identification while my prosody fingerprint stays flat at $\sim$ 28%; they cross over around a formant ratio of 0.76. It's a weaker biometric on clean audio, so it's a complement, not a replacement — and fused with the voiceprint it recovers accuracy exactly where the voiceprint fails. Honest caveat: my disguise is DSP resynthesis, a weaker stand-in for real neural cloning, and I've built the hook to rerun the whole protocol against RVC/XTTS on a GPU."

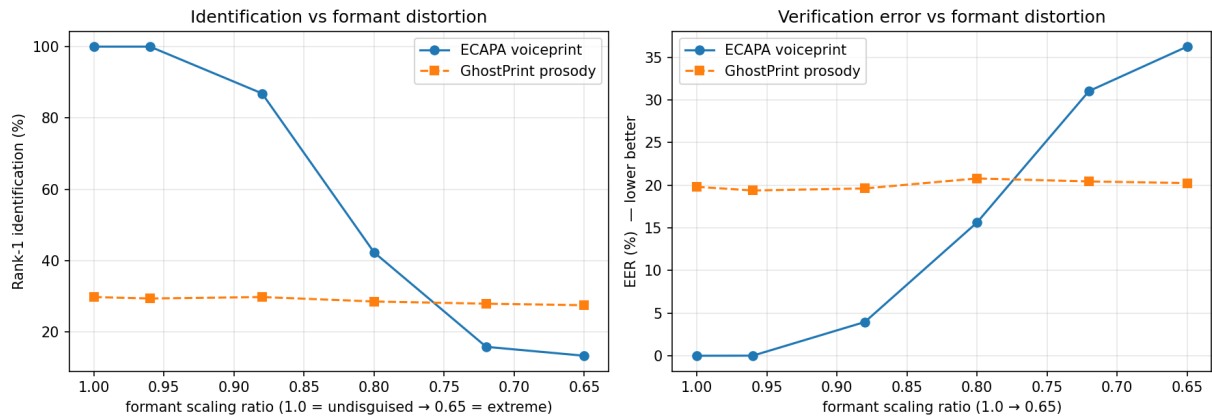
11. Map of the repo

Path	What it does
<code>ghostprint/config.py</code>	All knobs: disguise presets, splits, model dims, training params
<code>ghostprint/disguise.py</code>	PSOLA disguise + neural-VC hook
<code>ghostprint/features.py</code>	Clone-surviving prosody features (relative by construction)
<code>ghostprint/data.py</code>	LibriSpeech indexing / manifests
<code>ghostprint/model.py</code>	BiGRU prosody encoder + AAM-softmax head
<code>scripts/prepare_data.py</code>	Select utterances, synthesize disguises
<code>scripts/extract_features.py</code>	Prosody features + ECAPA baseline embeddings
<code>scripts/train.py</code>	Train the encoder with disguise augmentation
<code>scripts/evaluate.py</code>	Watchlist eval: EER / rank-1 / hit-rate, fusion, plots
<code>scripts/sweep.py</code>	Formant dose-response curve (headline result)
<code>scripts/ivr_experiment.py</code>	Simulated behavioral-channel re-ID
<code>demo.py</code>	Investigator two-call comparison

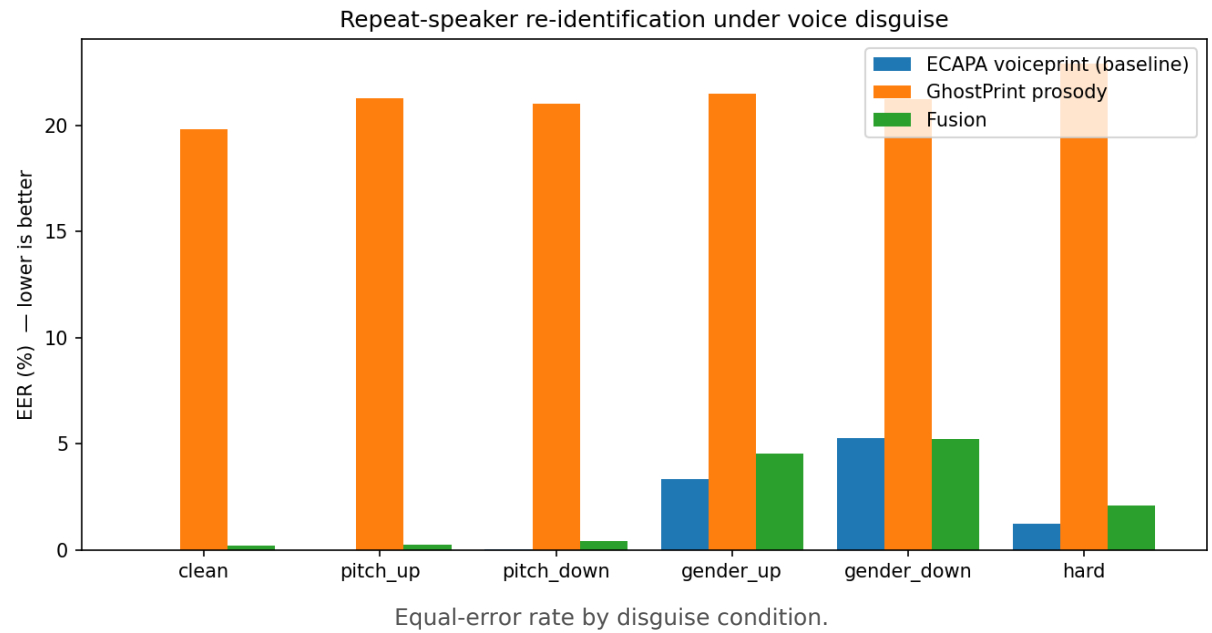
Path	What it does
README.md / RESULTS.md	Overview / full results writeup

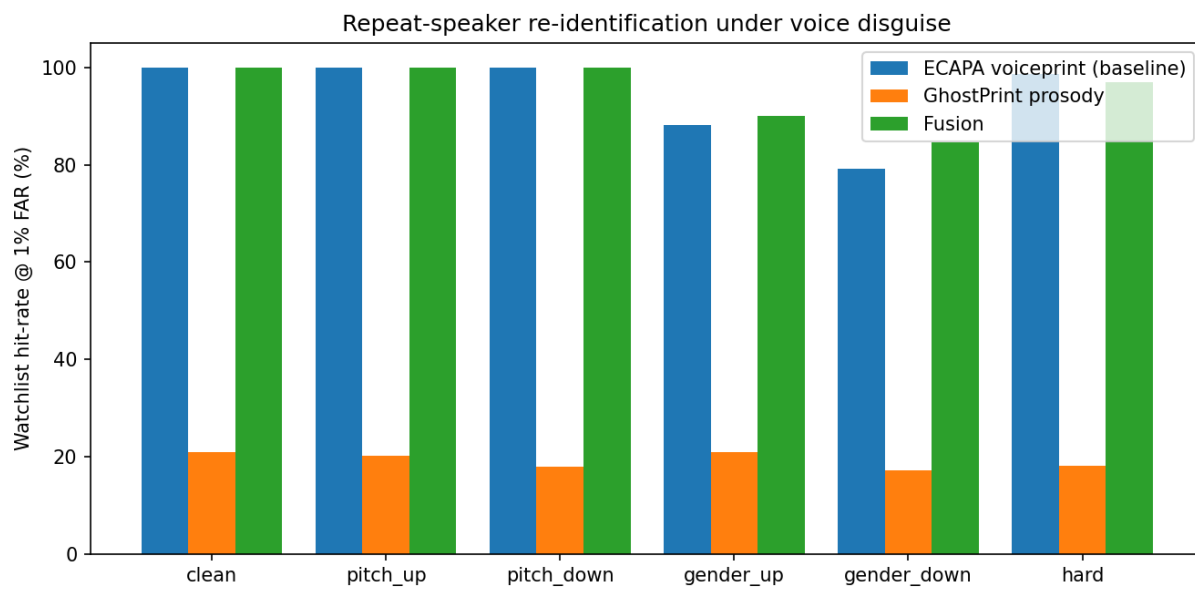
Appendix — Figures

Dose-response: voiceprint degrades under formant manipulation, prosody stays flat



Formant dose-response: the voiceprint (ECAPA) collapses as formants are distorted, while the GhostPrint prosody fingerprint stays flat.





Watchlist hit-rate (TPR at 1% FAR) by disguise condition.